

Phần 1: Tổng quan về Học có giám sát (Supervised Learning) và Bài toán Phân lớp

Mục tiêu học tập:

- Hiểu rõ khái niệm và quy trình của Học có giám sát.
- Phân biệt được đặc điểm của bài toán Phân lớp (Classification) so với bài toán Hồi quy (Regression).

1.1 Khái niệm cơ bản về Học có giám sát

Trong **Học có giám sát (Supervised Learning)**, thuật toán học máy được cung cấp một tập dữ liệu chứa các mẫu đã biết trước kết quả. Mỗi mẫu dữ liệu huấn luyện bao gồm một cặp:

- **Đầu vào x_i** : Là các vector đặc trưng (features) đại diện cho dữ liệu. Ví dụ: *Vector pixel của một bức ảnh, độ dài, chiều rộng của đối tượng.*
- **Đầu ra mục tiêu t_i** : Là nhãn (label) hoặc kết quả mong muốn tương ứng với đầu vào đó.

Mục tiêu tối thượng của mô hình là điều chỉnh các tham số bên trong nhằm tối đa hóa sự đồng thuận giữa dự đoán của nó và đầu ra mục tiêu thực tế. Về mặt toán học, điều này tương đương với việc cố gắng giảm thiểu hàm rủi ro hoặc tổn thất (loss/risk).

1.2 Phân biệt Bài toán Phân lớp và Hồi quy

Tùy thuộc vào bản chất của đầu ra mục tiêu t_i , các bài toán học có giám sát được phân thành hai nhóm chính:

1. **Hồi quy (Regression)**: Nhiệm vụ này được sử dụng khi đầu ra mục tiêu là các **giá trị liên tục** (vô hướng hoặc vector). Ví dụ điển hình: *Dự đoán giá bán của một ngôi nhà dựa trên diện tích, số phòng, khoảng cách di chuyển.*
2. **Phân lớp (Classification)**: Nhiệm vụ này được sử dụng khi đầu ra mục tiêu là các **nhãn rời rạc** thuộc về một tập hợp các lớp cho trước.
 - Trong ngữ cảnh Thị giác máy tính: Bài toán **Phân lớp ảnh ngữ nghĩa (Semantic Image Classification)** yêu cầu mô hình dự đoán và gán nhãn cho toàn bộ một bức ảnh vào một danh mục khả dĩ nhất (ví dụ: phân định ảnh đó chứa con ngựa, con mèo hay chiếc ô tô).

1.3 Quy trình 2 giai đoạn cốt lõi

Bất kỳ một hệ thống học có giám sát tiêu chuẩn nào cũng phải trải qua quy trình 2 giai đoạn sau:

- **Giai đoạn huấn luyện (Training phase):** Thuật toán xử lý lặp đi lặp lại toàn bộ tập dữ liệu huấn luyện (gồm các cặp đặc trưng và nhãn x_i, t_i) để học các quy luật ngầm, từ đó thiết lập và cố định mô hình.
- **Giai đoạn kiểm thử (Test phase):** Áp dụng mô hình đã được huấn luyện xong vào thực tế để dự đoán nhãn cho các dữ liệu đầu vào mới, chưa từng xuất hiện trong tập huấn luyện.

Nhiệm vụ: Sinh viên chạy đoạn mã Python dưới đây để khởi tạo dữ liệu mô phỏng sự khác biệt giữa Bài toán Phân lớp và Bài toán Hồi quy.

```
import numpy as np

# 1. MÔ PHỎNG BÀI TOÁN PHÂN LỚP (CLASSIFICATION)
# Giả sử X là đặc trưng của các loài động vật: [Cân nặng (kg), Chiều cao (cm)]
X_classification = np.array([
    [20, 50], # Mẫu 1: Chó
    [25, 55], # Mẫu 2: Chó
    [4, 25],  # Mẫu 3: Mèo
    [5, 28]   # Mẫu 4: Mèo
])

# Nhãn t_i là các giá trị RỜI RẠC (0 đại diện cho 'Chó', 1 đại diện cho 'Mèo')
T_classification = np.array([0, 0, 1, 1])

print("--- DỮ LIỆU BÀI TOÁN PHÂN LỚP ---")
print("Đặc trưng đầu vào X:\n", X_classification)
print("Nhãn mục tiêu T (Rời rạc):", T_classification)
print("="*40)

# 2. MÔ PHỎNG BÀI TOÁN HỒI QUY (REGRESSION)
# Giả sử X là đặc trưng của ngôi nhà: [Diện tích (m2), Khoảng cách đến trung tâm]
X_regression = np.array([
    [100, 2.5],
    [80, 5.0],
    [150, 1.2],
    [60, 10.0]
])

# Đầu ra t_i là các giá trị LIÊN TỤC (Đại diện cho giá nhà tính bằng Tỷ VNĐ)
T_regression = np.array([5.2, 3.1, 9.5, 1.8])

print("--- DỮ LIỆU BÀI TOÁN HỒI QUY ---")
```

```
print("Đặc trưng đầu vào X:\n", X_regression)
print("Đầu ra mục tiêu T (Liên tục):", T_regression)
```

```
--- DỮ LIỆU BÀI TOÁN PHÂN LỚP ---
Đặc trưng đầu vào X:
[[20 50]
 [25 55]
 [ 4 25]
 [ 5 28]]
Nhãn mục tiêu T (Rời rạc): [0 0 1 1]
=====
--- DỮ LIỆU BÀI TOÁN HỒI QUY ---
Đặc trưng đầu vào X:
[[100.    2.5]
 [ 80.    5. ]
 [150.    1.2]
 [ 60.   10. ]]
Đầu ra mục tiêu T (Liên tục): [5.2 3.1 9.5 1.8]
```

Phần 2: Phân lớp với K-Nearest Neighbors (KNN) và Tinh chỉnh Siêu tham số

Mục tiêu học tập:

- Nắm vững bản chất phi tham số (non-parametric) và cơ chế "học lười/vét cận" của KNN.
- Hiểu được sự ảnh hưởng của siêu tham số k đến ranh giới quyết định.
- Biết cách sử dụng Kiểm chứng chéo (Cross-validation) để tìm k tối ưu, khắc phục hiện tượng quá khớp (overfitting) và dưới khớp (underfitting).

2.1 Bản chất cốt lõi của K-Nearest Neighbors (KNN)

Thuật toán K-lân cận gần nhất (KNN) là một kỹ thuật học máy đơn giản nhưng cực kỳ mạnh mẽ thuộc nhóm **phi tham số (non-parametric technique)**. Đặc tính của thuật toán này bao gồm:

- **Không tóm tắt dữ liệu:** Thuật toán hoàn toàn không sử dụng một hàm giải tích tổng quát nào để mô hình hóa dữ liệu. Thay vào đó, **toàn bộ các mẫu huấn luyện đều được giữ lại nguyên vẹn trong bộ nhớ**.
- **Không có pha huấn luyện tường minh:** Quá trình huấn luyện gần như không làm gì cả. Mọi tính toán phức tạp chỉ thực sự diễn ra tại thời điểm dự đoán (pha kiểm thử).
- **Cơ chế vét cận (Brute-force) và Bầu chọn đa số:** Khi có một dữ liệu mới, hệ thống sẽ tính khoảng cách trực tiếp từ điểm này đến *tất cả* các mẫu đã lưu, tìm ra k

lân cận gần nhất và đưa ra phán quyết lớp dựa trên nhãn xuất hiện nhiều nhất (bầu chọn đa số).

2.2 Hiện tượng Quá khớp, Dưới khớp và Tinh chỉnh siêu tham số k

Việc lựa chọn giá trị k trực tiếp quyết định đến hình dáng của ranh giới quyết định (decision boundary):

- **Quá khớp (Overfitting) - Khi k quá nhỏ (ví dụ $k = 1$):** Mô hình phản ứng cực kỳ nhạy cảm với các điểm nhiễu ngẫu nhiên trong tập huấn luyện, khiến ranh giới quyết định bị phân mảnh.
- **Dưới khớp (Underfitting) - Khi k quá lớn:** Ranh giới quyết định bị làm mượt (over-smooths) quá mức, dẫn đến việc các vùng dữ liệu nhỏ bị "nuốt chửng" bởi các lớp đa số.
- **Giải pháp:** Chúng ta không thể đoán mò k . Cách tiêu chuẩn là sử dụng kỹ thuật **Kiểm chứng chéo (Cross-validation)** trên tập dữ liệu để dò tìm và đánh giá giá trị k mang lại độ chính xác tổng quát hóa cao nhất.

Nhiệm vụ: Sinh viên thực thi đoạn mã dưới đây. Đoạn mã này sử dụng thư viện `scikit-learn` để huấn luyện mô hình KNN trên một tập dữ liệu giả lập. Sinh viên sẽ quan sát được sự thay đổi của ranh giới quyết định khi thay đổi siêu tham số k .

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

# 1. TẠO DỮ LIỆU GIẢ LẬP GỒM 2 LỚP (CÓ CHỨA NHIỀU)
X, y = make_classification(n_samples=200, n_features=2, n_informative=2, n_redundant=1,
                           n_clusters_per_class=1, flip_y=0.1, random_state=42)

# 2. KHẢO SÁT ẢNH HƯỞNG CỦA K THÔNG QUA CROSS-VALIDATION
# Thử các giá trị k lẻ từ 1 đến 50
k_values = [k for k in range(1, 50, 2)]
cv_scores = []

print("--- ĐÁNH GIÁ ĐỘ CHÍNH XÁC THEO k (CROSS-VALIDATION) ---")
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    # Chia tập dữ liệu làm 5 phần (5-fold) để kiểm chứng chéo
    scores = cross_val_score(knn, X, y, cv=5, scoring='accuracy')
    mean_score = scores.mean()
    cv_scores.append(mean_score)
    print(f"k = {k:2d} | Độ chính xác Cross-validation = {mean_score:.4f}")
```

```

# Tìm ra k tối ưu nhất
optimal_k = k_values[np.argmax(cv_scores)]
print(f"\n=> SIÊU THAM SỐ TỐI ƯU: k = {optimal_k}")

# 3. TRỰC QUAN HÓA SO SÁNH RANH GIỚI QUYẾT ĐỊNH
# (Sinh viên tự tinh chỉnh mảng list_k để thấy rõ over-smooths và overfitting)
list_k = [1, optimal_k, 99]
plt.figure(figsize=(15, 4))

for i, k in enumerate(list_k, 1):
    plt.subplot(1, 3, i)
    knn = KNeighborsClassifier(n_neighbors=k).fit(X, y)

    # Vẽ ranh giới quyết định
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05), np.arange(y_min, y_max, 0.05))
    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)
    plt.title(f"KNN với k = {k}\n" +
              ("Quá khớp (Overfitting)" if k==1 else
               ("Dưới khớp (Underfitting)" if k==99 else "k Tối ưu")))

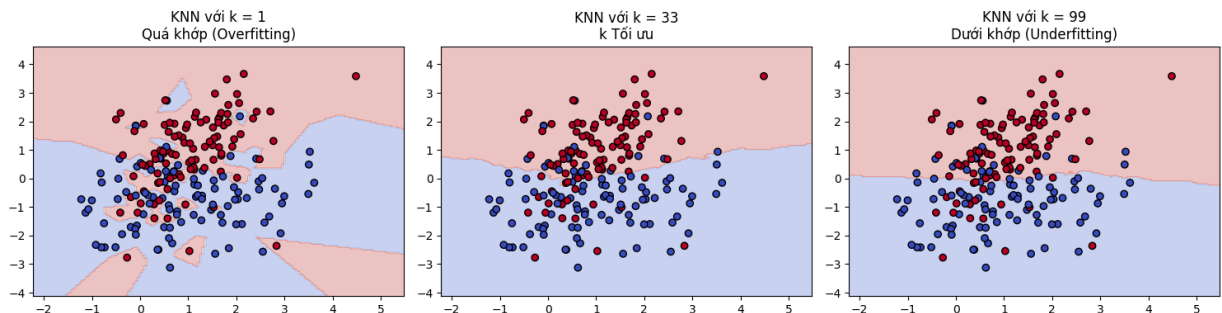
plt.tight_layout()
plt.show()

```

--- ĐÁNH GIÁ ĐỘ CHÍNH XÁC THEO k (CROSS-VALIDATION) ---

k = 1		Độ chính xác Cross-validation = 0.7150
k = 3		Độ chính xác Cross-validation = 0.7450
k = 5		Độ chính xác Cross-validation = 0.7450
k = 7		Độ chính xác Cross-validation = 0.7450
k = 9		Độ chính xác Cross-validation = 0.7450
k = 11		Độ chính xác Cross-validation = 0.7550
k = 13		Độ chính xác Cross-validation = 0.7450
k = 15		Độ chính xác Cross-validation = 0.7550
k = 17		Độ chính xác Cross-validation = 0.7800
k = 19		Độ chính xác Cross-validation = 0.7800
k = 21		Độ chính xác Cross-validation = 0.7800
k = 23		Độ chính xác Cross-validation = 0.7950
k = 25		Độ chính xác Cross-validation = 0.7950
k = 27		Độ chính xác Cross-validation = 0.7950
k = 29		Độ chính xác Cross-validation = 0.7950
k = 31		Độ chính xác Cross-validation = 0.7950
k = 33		Độ chính xác Cross-validation = 0.8050
k = 35		Độ chính xác Cross-validation = 0.7850
k = 37		Độ chính xác Cross-validation = 0.8000
k = 39		Độ chính xác Cross-validation = 0.7850
k = 41		Độ chính xác Cross-validation = 0.7900
k = 43		Độ chính xác Cross-validation = 0.7900
k = 45		Độ chính xác Cross-validation = 0.7800
k = 47		Độ chính xác Cross-validation = 0.7950
k = 49		Độ chính xác Cross-validation = 0.8000

=> SIÊU THAM SỐ TỐI ƯU: k = 33



Phần 3: Tăng tốc KNN với Lân cận gần nhất xấp xỉ (ANN), FLANN và Faiss

Mục tiêu học tập:

- Hiểu được điểm nghẽn hiệu năng của KNN vét cạn (Brute-force) khi áp dụng vào Thị giác máy tính.
- Nắm bắt cơ chế của kỹ thuật Tìm kiếm lân cận gần nhất xấp xỉ (ANN) và cấu trúc Cây K-d (K-d Trees).

- Thực hành sử dụng thư viện FLANN trong OpenCV và tìm hiểu nguyên lý của thư viện Faiss cho dữ liệu quy mô tỷ vector.

3.1 Điểm nghẽn hiệu năng của KNN nguyên thủy

Trong thực tế Thị giác máy tính, một hệ thống có thể chứa hàng trăm ngàn đến hàng triệu bức ảnh, mỗi bức ảnh lại trích xuất ra hàng ngàn vector đặc trưng nhiều chiều (ví dụ SIFT 128 chiều). Thuật toán KNN nguyên thủy yêu cầu phải tính toán khoảng cách từ điểm truy vấn tới **tất cả** các mẫu trong tập dữ liệu. Độ phức tạp tính toán tỷ lệ thuận với số lượng mẫu, khiến phương pháp vét cạn trở nên quá chậm và không thể đáp ứng yêu cầu thời gian thực.

3.2 Giải pháp: Lân cận gần nhất xấp xỉ (ANN) và Cây K-d

Để phá vỡ rào cản này, chúng ta sử dụng **Tim kiếm lân cận gần nhất xấp xỉ (Approximate Nearest Neighbors - ANN)**, chấp nhận đánh đổi một lượng nhỏ độ chính xác để lấy tốc độ gia tốc vượt trội.

Một trong những cấu trúc dữ liệu cốt lõi của ANN là **Cây K-d (K-d trees)**:

- **Cấu trúc phân tầng:** Không gian đặc trưng đa chiều được phân chia bằng cách đệ quy cắt các siêu mặt phẳng (hyperplanes) song song với các trục tọa độ đan xen nhau. Các nút trong (interior node) đóng vai trò là mặt phẳng cắt, còn nút lá (leaf node) chứa dữ liệu thực tế.
- **Chiến lược duyệt Best Bin First (BBF):** Khi truy vấn, hệ thống không duyệt toàn bộ cây một cách cứng nhắc mà ưu tiên duyệt các không gian (bins) có khoảng cách vật lý gần nhất với điểm truy vấn, giúp tìm ra lân cận cực kỳ nhanh chóng.

3.3 Công cụ thực tiễn: FLANN và Faiss

- **FLANN (Fast Library for Approximate Nearest Neighbors):** Là thư viện mạnh mẽ được tích hợp sâu vào OpenCV. FLANN chứa các thuật toán lõi như Cây K-d ngẫu nhiên (Randomized k-d trees), Cây K-means ưu tiên, và Băm nhảy cảm cục bộ (LSH). Đặc biệt, FLANN có tính năng tự động phân tích tập dữ liệu để chọn thuật toán và tham số tối ưu nhất.
- **Faiss (Facebook AI Similarity Search):** Khi quy mô dữ liệu phình to lên tới **hàng tỷ vector**, FLANN hay LSH sẽ gặp giới hạn về bộ nhớ. Thư viện Faiss giải quyết vấn đề này bằng sức mạnh xử lý song song của GPU và công nghệ **Lượng hóa tích (Product Quantization)**, giúp nén các vector khổng lồ thành mã nhị phân nhỏ gọn.

Nhiệm vụ: Sinh viên chạy đoạn mã dưới đây để tạo ra một tập dữ liệu giả lập gồm 100,000 vector đặc trưng (mỗi vector 128 chiều, tương tự SIFT). Sau đó, chúng ta sẽ đo

lượng và so sánh thời gian tìm kiếm lân cận giữa phương pháp vét cạn (**BFMatcher**) và phương pháp xấp xỉ (**FlannBasedMatcher**) của OpenCV.

```
import cv2
import numpy as np
import time

# 1. TẠO DỮ LIỆU GIẢ LẬP
NUM_SAMPLES = 100000 # 100 ngàn vector đặc trưng huấn luyện
NUM_QUERIES = 1000   # 1 ngàn vector cần dự đoán (kiểm thử)
DIM = 128             # Số chiều của mỗi vector (giống SIFT)

print(f"Đang khởi tạo dữ liệu: {NUM_SAMPLES} mẫu huấn luyện và {NUM_QUERIES} mẫu")
# Dữ liệu OpenCV yêu cầu định dạng float32
train_features = np.random.random((NUM_SAMPLES, DIM)).astype(np.float32)
query_features = np.random.random((NUM_QUERIES, DIM)).astype(np.float32)

# =====
# 2. TÌM KIẾM VẾT CẠN (BRUTE-FORCE MATCHER)
# =====
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)

start_time = time.time()
# Tìm 2 lân cận gần nhất (k=2) cho mỗi điểm truy vấn
matches_bf = bf.knnMatch(query_features, train_features, k=2)
bf_time = time.time() - start_time

print(f" Brute-Force (Vết cạn) mất: {bf_time:.4f} giây")

# =====
# 3. TÌM KIẾM XẤP XỈ VỚI FLANN (K-D TREES)
# =====
# Cấu hình tham số cho thuật toán Randomized K-d Trees
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
# checks: Số lần duyệt cây (càng lớn càng chính xác nhưng chậm hơn)
search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)

start_time = time.time()
# Tìm 2 lân cận gần nhất (k=2) bằng ANN
matches_flann = flann.knnMatch(query_features, train_features, k=2)
flann_time = time.time() - start_time

print(f" FLANN (Approximate) mất : {flann_time:.4f} giây")
print(f"=> FLANN nhanh hơn Brute-Force khoảng {bf_time / flann_time:.2f} lần!")

# =====
# 4. MỞ RỘNG VỚI FAISS (Sử dụng Product Quantization)
```



```
# LƯU Ý: Đoạn mã dưới đây mang tính chất giới thiệu cấu trúc Faiss.
# =====
# import faiss
# index_flat = faiss.IndexFlatL2(DIM)
# quantizer = faiss.IndexFlatL2(DIM)
# index_pq = faiss.IndexIVFPQ(quantizer, DIM, nlist=100, m=8, 8)
# index_pq.train(train_features)
# index_pq.add(train_features)
# distances, indices = index_pq.search(query_features, 2)
```

Đang khởi tạo dữ liệu: 100000 mẫu huấn luyện và 1000 mẫu truy vấn...
 Brute-Force (Vét cạn) mất: 7.1684 giây
 FLANN (Approximate) mất : 1.0805 giây
 => FLANN nhanh hơn Brute-Force khoảng 6.63 lần!

Phần 4: Giải quyết sự bất đồng nhất đặc trưng với Mô hình Bag-of-Features (BoF) (Bước 1 & 2)

Mục tiêu học tập:

- Hiểu rõ "Nghịch lý kích thước" khi trích xuất đặc trưng và nguồn gốc của mô hình Bag-of-Words từ lĩnh vực Xử lý ngôn ngữ tự nhiên (NLP).
- Nắm bắt khái niệm "Từ vựng thị giác" (Visual Words) và bản chất "chiếc túi" (bỏ qua thông tin không gian).
- Thực hành Bước 1 (Trích xuất đặc trưng SIFT/SURF) và Bước 2 (Gom cụm K-means để học Từ điển thị giác).

4.1 Vấn đề "Cổ chai" và Nguồn gốc ý tưởng từ NLP

- **Nghịch lý Kích thước:** Các thuật toán trích xuất đặc trưng cục bộ (như SIFT, SURF) sẽ tìm ra các điểm nổi bật trên ảnh. Tuy nhiên, một bức ảnh phong cảnh phức tạp có thể sinh ra 800 điểm đặc trưng, trong khi một bức ảnh đơn giản chỉ sinh ra 500 điểm. Các bộ phân lớp "não bộ" như KNN hay SVM không thể so sánh trực tiếp hai ma trận dữ liệu có kích thước khác biệt nhau.
- **Giải pháp - Mô hình "Túi" (Bag):** Kế thừa từ Xử lý ngôn ngữ tự nhiên (NLP), mô hình Bag-of-Features xem một bức ảnh giống như một "chiếc túi" chứa các "từ vựng thị giác".
- **Giả định độc lập:** Mô hình này chấp nhận đánh đổi, **bỏ qua hoàn toàn mối quan hệ không gian** và cấu trúc hình học của vật thể. Nó không quan tâm "bánh xe" nằm dưới "thân xe" hay không, mà chỉ đếm xem có bao nhiêu "bánh xe" xuất hiện trong túi.

4.2 Pipeline BoF: Bước 1 & Bước 2

Quy trình xây dựng mô hình BoF gồm 4 bước, trong phần này ta tập trung vào 2 bước đầu tiên:

- **Bước 1 - Trích xuất đặc trưng (Feature Extraction):** Quét qua toàn bộ ảnh huấn luyện để thu thập hàng triệu vector đặc trưng cục bộ (ví dụ: vector SIFT 128 chiều).
- **Bước 2 - Học Từ vựng Thị giác (Learning Visual Vocabulary):**
 - Do các vector trích xuất ra là các giá trị liên tục và lộn xộn, ta cần gom nhóm chúng lại.
 - Sử dụng thuật toán học không giám sát **K-means Clustering** để gom cụm toàn bộ các vector này thành K nhóm.
 - Mỗi **tâm cụm (cluster center)** được xem là một **"Từ vựng thị giác" (Visual Word)** chuẩn hóa. Tập hợp K tâm cụm này tạo thành một cuốn **Từ điển (Codebook / Dictionary)**.

Nhiệm vụ: Đoạn mã Python dưới đây mô phỏng Bước 1 và Bước 2. Chúng ta sẽ tạo ra các điểm đặc trưng SIFT giả lập với số lượng ngẫu nhiên cho nhiều bức ảnh khác nhau. Sau đó, ta gộp tất cả lại và dùng `KMeans` để xây dựng "Từ điển Thị giác".

```
import numpy as np
from sklearn.cluster import KMeans

# =====
# BƯỚC 1: TRÍCH XUẤT ĐẶC TRƯNG (MÔ PHỎNG SIFT 128 CHIỀU)
# =====
np.random.seed(42)
DIM = 128 # SIFT tạo ra vector 128 chiều

# Giả sử ta có 3 bức ảnh với số lượng điểm đặc trưng hoàn toàn khác nhau
img1_features = np.random.rand(500, DIM) # Ảnh 1 có 500 điểm
img2_features = np.random.rand(800, DIM) # Ảnh 2 có 800 điểm
img3_features = np.random.rand(350, DIM) # Ảnh 3 có 350 điểm

print(f"Kích thước đặc trưng Ảnh 1: {img1_features.shape}")
print(f"Kích thước đặc trưng Ảnh 2: {img2_features.shape}")
print(f"Kích thước đặc trưng Ảnh 3: {img3_features.shape}")
print("-> Bất đồng nhất kích thước: Không thể đưa trực tiếp vào SVM/KNN!\n")

# Gom TẤT CẢ đặc trưng cục bộ từ TẤT CẢ các ảnh huấn luyện vào một "kho" chung
all_training_features = np.vstack((img1_features, img2_features, img3_features))
print(f"Tổng số vector đặc trưng thu thập được: {all_training_features.shape} ve

# =====
# BƯỚC 2: HỌC TỪ VỰNG THỊ GIÁC (TẠO CODEBOOK BẰNG K-MEANS)
# =====
# Định nghĩa kích thước Từ điển (số lượng Từ vựng thị giác K)
```

```
K_DICT_SIZE = 50
```

```
print(f"\nĐang chạy K-means để tìm {K_DICT_SIZE} Từ vựng thị giác...")
# Khởi tạo và huấn luyện mô hình K-means
kmeans = KMeans(n_clusters=K_DICT_SIZE, random_state=42, n_init=10)
kmeans.fit(all_training_features)

# Các tâm cụm (cluster centers) chính là các Từ vựng thị giác
visual_dictionary = kmeans.cluster_centers_

print("Hoàn tất học Từ điển!")
print(f"Kích thước của Từ điển (Codebook): {visual_dictionary.shape}")
print(f"-> Bây giờ ta đã có {K_DICT_SIZE} từ vựng chuẩn (mỗi từ dài {DIM} chiều)
```

Kích thước đặc trưng Ảnh 1: (500, 128)

Kích thước đặc trưng Ảnh 2: (800, 128)

Kích thước đặc trưng Ảnh 3: (350, 128)

-> Bất đồng nhất kích thước: Không thể đưa trực tiếp vào SVM/KNM!

Tổng số vector đặc trưng thu thập được: (1650, 128) vectors

Đang chạy K-means để tìm 50 Từ vựng thị giác...

Hoàn tất học Từ điển!

Kích thước của Từ điển (Codebook): (50, 128)

-> Bây giờ ta đã có 50 từ vựng chuẩn (mỗi từ dài 128 chiều) để đại diện cho mọi

Phần 5: Lượng hóa, Biểu đồ tần suất và Khớp Kim tự tháp Không gian (SPM)

Mục tiêu học tập:

- Hoàn thiện quy trình BoF với Bước 3 (Lượng hóa) và Bước 4 (Lập biểu đồ).
- Thấy được sức mạnh của BoF: Biến đổi ảnh có kích thước bất kỳ thành một vector duy nhất có độ dài cố định.
- Hiểu điểm mù của BoF và cách Khớp Kim tự tháp Không gian (SPM) giải quyết bài toán mất thông tin không gian.

5.1 Bước 3: Lượng hóa đặc trưng (Feature Quantization)

- **Mục tiêu:** Rời rạc hóa (discretize) các vector đặc trưng liên tục thành các ID từ vựng cụ thể.
- **Cơ chế:** Khi có một bức ảnh mới, ta trích xuất các vector đặc trưng (ví dụ SIFT 128D) của nó. Sau đó, đo khoảng cách (Euclidean hoặc Mahalanobis) từ mỗi vector này tới tất cả các tâm cụm trong Từ điển (đã học ở Bước 2). Vector đặc trưng sẽ bị

thay thế hoàn toàn bởi ID của **từ vựng thị giác gần nhất**. Sự phức tạp 128 chiều nay được nén lại thành một mã định danh duy nhất.

5.2 Bước 4: Lập biểu đồ tần suất (Histogram Representation)

- Sau khi lượng hóa, hệ thống quét qua toàn bộ bức ảnh và đếm số lần xuất hiện của từng ID từ vựng thị giác.
- Kết quả thu được là một biểu đồ Histogram có độ dài cố định, **hoàn toàn bằng với kích thước của Từ điển K** .
- **Ý nghĩa cốt lõi:** Bất chấp ảnh ban đầu chứa 500 hay 5.000 điểm SIFT, đầu ra cuối cùng luôn là một vector duy nhất có chiều dài K . Vector này chính là "bộ khung xương" dữ liệu hoàn hảo để đưa vào các bộ phân lớp như SVM.

Nhiệm vụ: Sinh viên chạy đoạn mã dưới đây để thực hiện Bước 3 và 4. Chú ý cách chúng ta tái sử dụng mô hình `kmeans` (Từ điển) từ Phần 4 để lượng hóa một bức ảnh kiểm thử hoàn toàn mới.

```
import numpy as np
import matplotlib.pyplot as plt

# Giả sử ta có 1 ảnh mới cần dự đoán (Ảnh Kiểm thử) trích xuất được 420 điểm SIFT
np.random.seed(99)
new_image_features = np.random.rand(420, DIM) # DIM = 128

print(f"Bức ảnh mới có {new_image_features.shape} điểm đặc trưng cục bộ.")

# =====
# BƯỚC 3: LƯỢNG HÓA ĐẶC TRƯNG (QUANTIZATION)
# =====
# Sử dụng kmeans.predict() để tìm ID tâm cụm gần nhất cho từng điểm SIFT
# Hàm predict ngầm thực hiện việc tính khoảng cách Euclidean tới K tâm cụm
visual_word_ids = kmeans.predict(new_image_features)

print("\n--- KẾT QUẢ BƯỚC 3 ---")
print(f"5 điểm SIFT đầu tiên được gán cho các Từ vựng ID: {visual_word_ids[:5]}")
print(f"Tổng số ID thu được: {len(visual_word_ids)} (bằng đúng số điểm SIFT)")

# =====
# BƯỚC 4: LẬP BIỂU ĐỒ TẦN SUẤT (HISTOGRAM)
# =====
# Đếm tần suất xuất hiện của các ID từ 0 đến K_DICT_SIZE - 1
bof_histogram, _ = np.histogram(visual_word_ids, bins=np.arange(K_DICT_SIZE + 1))

# Chuẩn hóa Histogram (L1 Normalization) để loại bỏ yếu tố kích thước ảnh
bof_histogram_normalized = bof_histogram / np.sum(bof_histogram)

print("\n--- KẾT QUẢ BƯỚC 4 ---")
```

```

print(f"Kích thước Vector Histogram sinh ra: {bof_histogram.shape} (Luôn luôn cố

# Trực quan hóa Biểu đồ (Túi từ vựng)
plt.figure(figsize=(10, 4))
plt.bar(range(K_DICT_SIZE), bof_histogram_normalized, color='orange', edgecolor=
plt.title("Bag-of-Features: Biểu đồ Tần suất Từ vựng Thị giác của Ảnh Kiểm thử")
plt.xlabel("ID Từ vựng (0 -> K-1)")
plt.ylabel("Tần suất xuất hiện (Đã chuẩn hóa)")
plt.show()

```

Bức ảnh mới có (420, 128) điểm đặc trưng cục bộ.

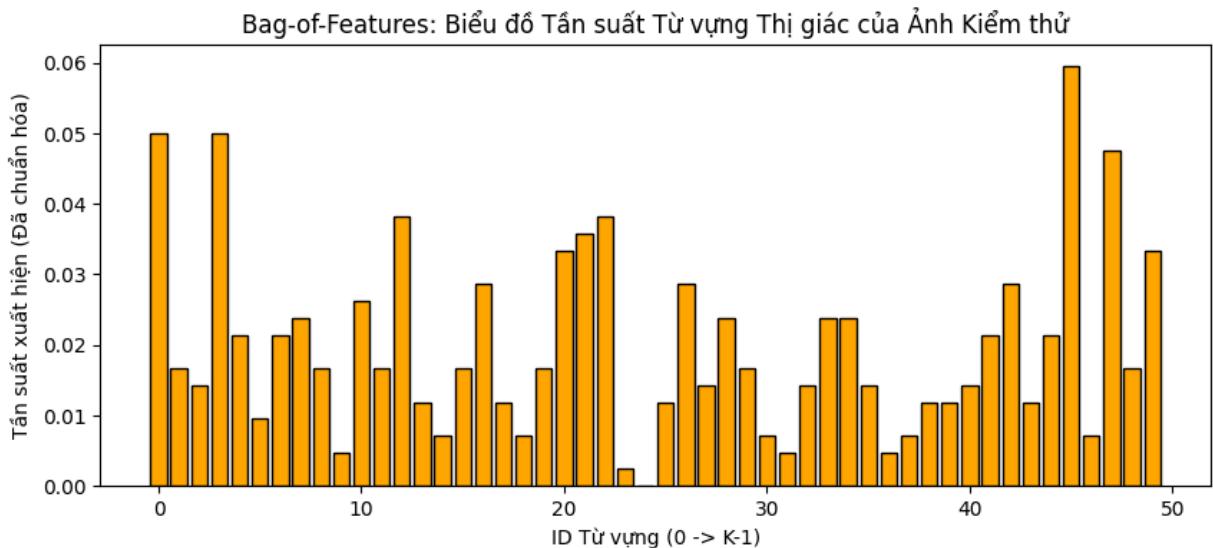
--- KẾT QUẢ BƯỚC 3 ---

5 điểm SIFT đầu tiên được gán cho các Từ vựng ID: [41 4 38 11 39]

Tổng số ID thu được: 420 (bằng đúng số điểm SIFT)

--- KẾT QUẢ BƯỚC 4 ---

Kích thước Vector Histogram sinh ra: (50,) (Luôn luôn cố định là K=50)



✓ 5.3 Vượt qua giới hạn với Khớp Kim tự tháp Không gian (SPM)

- **Điểm mù của mô hình BoF:** Bản chất "Túi" (Bag) dựa trên giả định độc lập, nó **bỏ qua hoàn toàn cấu trúc hình học và vị trí không gian** của vật thể. Máy tính biết có 4 "bánh xe" và 2 "cửa sổ", nhưng không biết bánh xe có nằm dưới cửa sổ hay không.
- **Giải pháp SPM (Spatial Pyramid Matching):** Được đề xuất bởi Lazebnik et al. (2006) để phục hồi lại khái niệm vị trí không gian 2D lỏng lẻo.
- **Cơ chế hoạt động:**
 1. Thay vì chỉ đếm Histogram trên toàn bộ ảnh (1×1), ảnh được chia thành các lưới không gian nhỏ dần: 2×2 (4 ô), 4×4 (16 ô)....
 2. Đếm tần suất cục bộ bên trong từng ô lưới.

3. Nối (concatenate) tất cả các Histogram này lại với nhau, tạo ra một biểu diễn không gian cực kỳ mạnh mẽ.

Nhiệm vụ: Sinh viên quan sát logic nhân bản độ dài vector của phương pháp SPM ở cấp độ $L = 1$ (lưới 2×2).

```
# MÔ PHỎNG KHỚP KIM TỰ THÁP KHÔNG GIAN (SPM) - CẤP ĐỘ L=1 (Lưới 2x2)

# Giả sử ảnh được chia thành 4 vùng (Trái-Trên, Phải-Trên, Trái-Dưới, Phải-Dưới)
# Mỗi vùng sinh ra một BoF Histogram độc lập độ dài K
hist_top_left = np.random.rand(K_DICT_SIZE)
hist_top_right = np.random.rand(K_DICT_SIZE)
hist_bottom_left = np.random.rand(K_DICT_SIZE)
hist_bottom_right = np.random.rand(K_DICT_SIZE)

# Nối các Histogram lại với nhau
spm_vector = np.concatenate([
    bof_histogram_normalized, # Mức L=0 (Toàn ảnh - K chiều)
    hist_top_left, hist_top_right, hist_bottom_left, hist_bottom_right # Mức L=1
])

print(f"Kích thước Vector gốc (L=0) : {bof_histogram_normalized.shape}")
print(f"Kích thước Vector SPM (L=0,1): {spm_vector.shape}")
print(f"-> Thấy rõ: Vector SPM đã dài gấp 5 lần (K + 4*K), chứa đựng thêm cấu trúc 2D")

Kích thước Vector gốc (L=0) : (50,)
Kích thước Vector SPM (L=0,1): (250,)
-> Thấy rõ: Vector SPM đã dài gấp 5 lần (K + 4*K), chứa đựng thêm cấu trúc 2D
```

Phần 6: Phân lớp Lề tối đa với Support Vector Machine (SVM)

Mục tiêu học tập:

- Hiểu nền tảng toán học của Bộ phân lớp lề tối đa (Maximum Margin Classifier) và khái niệm Vector hỗ trợ (Support Vectors).
- Nắm được cách xử lý dữ liệu nhiễu/chồng lấn bằng Lề mềm (Soft Margin) và hàm mất mát Hinge Loss.
- Giải quyết dữ liệu phi tuyến tính bằng Thủ thuật Hạt nhân (Kernel Trick).
- Hiểu chiến lược áp dụng SVM cho bài toán Đa lớp (Multi-class) bằng OVO.

6.1 Triết lý Lề Tối đa (Maximum Margin) và Vector Hỗ trợ (Support Vectors)

- **Bản chất phân biệt:** SVM là thuật toán học có giám sát tập trung trực tiếp vào việc tìm cách phân định sự khác biệt giữa các lớp dữ liệu.
- **Lề tối đa:** Đối với hai lớp dữ liệu có thể phân tách tuyến tính, SVM không chọn một siêu mặt phẳng (đường phân tách) ngẫu nhiên. Nó chọn siêu mặt phẳng tạo ra **khoảng cách (lề - margin) lớn nhất** tới các điểm dữ liệu gần nhất của cả hai lớp. Việc tối đa hóa lề giúp giảm thiểu rủi ro dự đoán sai khi gặp dữ liệu kiểm thử mới.
- **Vector hỗ trợ:** Ranh giới quyết định của SVM cực kỳ thanh thoát vì nó **không phụ thuộc vào toàn bộ dữ liệu**. Nó chỉ được định hình bởi một số ít các điểm dữ liệu nằm ngay trên ranh giới lề, gọi là các "Vector hỗ trợ".

6.2 Lề Mềm (Soft Margin) và Hàm Mất mát Bản lề (Hinge Loss)

- Trong thực tế thị giác máy tính, dữ liệu giữa các lớp thường chồng lấn (overlapping) lên nhau, khiến một ranh giới cứng (hard margin) thất bại.
- **Giải pháp:** SVM cho phép một số điểm dữ liệu vi phạm lề (nằm sai phía) thông qua Lề mềm. Nó sử dụng hàm mất mát **Hinge Loss** (chỉ phạt tăng dần đối với những mẫu vi phạm lề, còn mẫu đúng phía sẽ có mức phạt bằng 0). Bài toán tối ưu lúc này là cực tiểu hóa tổng của Hinge Loss và một thành phần chuẩn hóa.

6.3 Xử lý Dữ liệu Phi tuyến với Thủ thuật Hạt nhân (Kernel Trick)

- Nếu hai lớp dữ liệu lồng ghép phức tạp (ví dụ dạng vòng tròn), siêu mặt phẳng tuyến tính sẽ hoàn toàn bất lực.
- **Thủ thuật Kernel Trick:** SVM nhân trọng số với giá trị của các hàm hạt nhân (kernel functions) có tâm đặt tại các điểm dữ liệu huấn luyện, ngầm ánh xạ dữ liệu lên một **không gian chiều cao hơn**, nơi chúng có thể bị cắt bởi một mặt phẳng phẳng.
- Một trong những hàm Kernel mạnh nhất là **RBF (Hàm cơ sở xuyên tâm / Gaussian)**, cho phép tạo ra các đường đồng mức cong khép kín bao quanh các vector hỗ trợ.

6.4 Phân lớp Đa lớp (Multi-class SVM)

- Bản chất toán học của SVM chỉ dành cho phân lớp nhị phân (0 hoặc 1). Để phân loại n lớp đối tượng (ví dụ: Chó, Mèo, Chim, Cá...), hệ thống sử dụng chiến lược **One-vs-One (OVO)**: Phân rã bài toán thành $\frac{n(n-1)}{2}$ bộ phân lớp nhị phân nhỏ và tổng hợp kết quả qua bầu chọn đa số (Voting).

Nhiệm vụ: Sinh viên chạy đoạn mã dưới đây để khảo sát khả năng phân tách của SVM. Đoạn mã giả lập một tập dữ liệu phi tuyến tính (dạng hai vòng tròn lồng nhau) và so sánh trực diện giữa hai mô hình: **Linear SVM (Tuyến tính)** và **RBF SVM (Phi tuyến)**. Sinh viên hãy quan sát các điểm được đánh dấu vòng tròn đen – đó chính là các Support Vectors.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.svm import SVC

# 1. TẠO DỮ LIỆU PHI TUYẾN TÍNH (Hai vòng tròn lồng nhau)
X, y = make_circles(n_samples=300, noise=0.1, factor=0.3, random_state=42)

# Khởi tạo 2 mô hình SVM: Tuyến tính và Phi tuyến (RBF)
# C là siêu tham số kiểm soát Lề mềm (Mức độ phạt khi phân loại sai)
svm_linear = SVC(kernel='linear', C=1.0)
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='auto')

models = [svm_linear, svm_rbf]
titles = ['Linear SVM (Thất bại với dữ liệu phi tuyến)',
          'RBF Kernel SVM (Phân tách thành công nhờ ánh xạ chiều cao)']

plt.figure(figsize=(14, 5))

# 2. HUẤN LUYỆN VÀ TRỰC QUAN HÓA RANH GIỚI
for i, model in enumerate(models):
    plt.subplot(1, 2, i + 1)

    # Huấn luyện mô hình
    model.fit(X, y)

    # Vẽ các điểm dữ liệu
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, s=30, edgecolors='k')

    # Vẽ các Vector hỗ trợ (Support Vectors) bằng vòng tròn đen
    plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
                s=100, facecolors='none', edgecolors='k', linewidths=1.5,
                label='Support Vectors')

    # Vẽ ranh giới quyết định (Decision Boundary)
    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
    xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 50),
                          np.linspace(ylim[0], ylim[1], 50))
    Z = model.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # Vẽ đường Lề (Margin) và ranh giới
    ax.contour(xx, yy, Z, colors='k', levels=[-1, 0, 1], alpha=0.5,
```



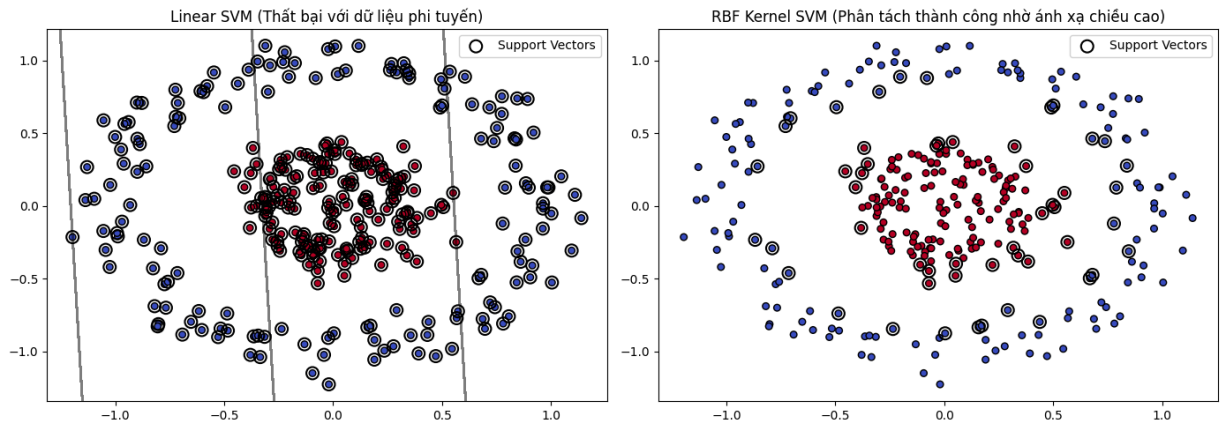
```

linestyles=['--', '-', '--'])

plt.title(titles[i])
plt.legend(loc='upper right')

plt.tight_layout()
plt.show()

```



Phần 7: Dự án Thực chiến: Nhận diện Người đi bộ (Pedestrian Detection) với HOG + SVM

Mục tiêu học tập:

- Vận dụng kết hợp kiến thức về đặc trưng thiết kế thủ công (Hand-crafted features) và bộ não phân lớp (Classifier).
- Tái hiện lại "Tiêu chuẩn vàng" (Gold Standard) do Dalal & Triggs đề xuất năm 2005: Sự kết hợp giữa **HOG (Histogram of Oriented Gradients)** và **Linear SVM**.
- Nắm vững quy trình 5 bước cốt lõi của một hệ thống nhận diện thực tế.
- Thực hành sử dụng thư viện OpenCV để khởi tạo bộ quét và nạp trọng số SVM đã huấn luyện sẵn.

7.1 Tại sao lại là bộ đôi HOG + Linear SVM?

- **Sức mạnh của HOG:** Thuật toán HOG bóc tách và loại bỏ hoàn toàn thông tin về màu sắc, chỉ giữ lại "bộ xương" (cấu trúc hình học, đường bao cơ thể) của vật thể. Do cấu trúc hình dáng của con người tương đối ổn định khi đứng thẳng, HOG tạo ra một biểu diễn cực kỳ xuất sắc.
- **Sự phù hợp với Linear SVM:** Đối với một cửa sổ trượt chứa người đi bộ, HOG sinh ra một vector đặc trưng dày đặc có độ dài lên tới 3.780 chiều. Không gian đa chiều khổng lồ này chính là môi trường lý tưởng để Linear SVM có thể dễ dàng tìm ra siêu mặt phẳng phân tách tuyến tính giữa lớp "Người" và "Không phải người".

7.2 Pipeline Hệ thống Nhận diện 5 Bước

Hệ thống nhận diện người đi bộ của chúng ta sẽ hoạt động ngầm qua 5 bước cốt lõi sau:

1. **Tiền xử lý (Preprocessing):** Chuẩn hóa gamma và màu sắc của ảnh đầu vào.
2. **Tính Gradient:** Tính toán độ lớn và hướng gradient tại từng pixel.
3. **Bỏ phiếu Không gian (Spatial Voting):** Chia ảnh thành các ô lưới (cells), gom nhóm thành các khối (blocks) trượt chồng lấp và lập biểu đồ Histogram.
4. **Chuẩn hóa Khối (Block Normalization):** Giúp đặc trưng bất biến với các điều kiện chiếu sáng khác nhau (ví dụ: ảnh bị bóng đổ hoặc phơi sáng quá mức).
5. **Phân loại với SVM:** Đưa vector khổng lồ 3.780 chiều thu được vào bộ phân lớp Linear SVM để đưa ra phán quyết cuối cùng.

7.3 Thực chiến Lập trình với OpenCV

Trong thực tế phát triển phần mềm (Góc nhìn Developer), chúng ta không cần viết lại thuật toán tối ưu toán học của SVM từ con số 0. Thư viện OpenCV đã cung cấp sẵn bộ mô tả HOG và cho phép nạp trực tiếp bộ trọng số SVM (đã được huấn luyện bởi hàng ngàn ảnh người đi bộ) vào hệ thống.

Nhiệm vụ: Sinh viên chạy đoạn mã dưới đây để kích hoạt hệ thống nhận diện người đi bộ. Hệ thống sẽ dùng một "cửa sổ trượt" đa tỷ lệ quét qua toàn bộ bức ảnh để tìm kiếm.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import requests
from PIL import Image
from io import BytesIO

# 1. TẢI ẢNH MẪU TỪ INTERNET
# Sử dụng một ảnh đường phố có người đi bộ để thực hành
url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/basketball.jpg"
response = requests.get(url)
image = np.array(Image.open(BytesIO(response.content)))

# Chuyển sang định dạng BGR để OpenCV xử lý nếu cần, ở đây PIL mở ra RGB nên ta cần chuyển
# Nếu dùng cv2.imread thì cần cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

print("1. Đang khởi tạo Bộ mô tả đặc trưng hình học (Mắt quét) - HOG Descriptor.")
hog = cv2.HOGDescriptor()

print("2. Đang nạp trọng số của Não bộ phán quyết (Pre-trained Linear SVM)...")
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

print("3. Đang tiến hành quét cửa sổ trượt đa tỷ lệ để tìm mục tiêu...")
```

```

print('3. Đang tiến hành quá trình trích xuất đặc trưng...')
# winStride: Bước nhảy của cửa sổ trượt (càng nhỏ càng kỹ nhưng chậm)
# padding: Vùng đệm xung quanh cửa sổ
# scale: Hệ số thay đổi kích thước ảnh để tìm người ở các xa gần khác nhau
(rects, weights) = hog.detectMultiScale(image, winStride=(8, 8), padding=(8, 8),

```

4. VẼ BOUNDING BOX VÀ HIỂN THỊ KẾT QUẢ

```

image_result = image.copy()
for (x, y, w, h) in rects:
    ... # Vẽ hình chữ nhật màu xanh lá
    ... cv2.rectangle(image_result, (x, y), (x + w, y + h), (0, 255, 0), 2)
    ... cv2.putText(image_result, 'Person', (x, y - 10),
    ..... cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

```

```

print(f"=> Đã tìm thấy {len(rects)} đối tượng người trong ảnh.")

```

```

# Trực quan hóa kết quả
plt.figure(figsize=(12, 8))
plt.imshow(image_result)
plt.title("Dự án Thực chiến: Nhận diện Người đi bộ bằng HOG + Linear SVM")
plt.axis('off')
plt.show()

```

1. Đang khởi tạo Bộ mô tả đặc trưng hình học (Mắt quét) - HOG Descriptor...
 2. Đang nạp trọng số của Não bộ phán quyết (Pre-trained Linear SVM)...
 3. Đang tiến hành quét cửa sổ trượt đa tỷ lệ để tìm mục tiêu...
- => Đã tìm thấy 2 đối tượng người trong ảnh.

Dự án Thực chiến: Nhận diện Người đi bộ bằng HOG + Linear SVM



✓ 7.4 Nâng cấp: Nhận diện người đi bộ trên Video

Quy trình xử lý video về bản chất là lặp lại việc xử lý ảnh trên từng khung hình (frame). Tuy nhiên, vì video có số lượng khung hình rất lớn, chúng ta cần tối ưu hóa tham số `winStride` và `scale` để đảm bảo tốc độ xử lý.

```
import cv2
import requests
import os
from google.colab.patches import cv2_imshow

# 1. TẢI VIDEO MẪU
video_url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/sample_video.avi"
video_path = "sample_video.avi"

print("Đang tải video mẫu...")
try:
```

```

response = requests.get(video_url, timeout=15)
if response.status_code == 200:
    with open(video_path, "wb") as f:
        f.write(response.content)
    print(f"Tải video thành công: {video_path} ({os.path.getsize(video_path)} bytes)")
else:
    print(f"Lỗi HTTP: {response.status_code}. Không thể tải video.")
except Exception as e:
    print(f"Lỗi kết nối: {e}")

# 2. KHỞI TẠO HOG + SVM
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

# 3. MỞ VIDEO VÀ XỬ LÝ
if os.path.exists(video_path) and os.path.getsize(video_path) > 0:
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Lỗi: OpenCV không thể giải mã video này.")
    else:
        frame_count = 0
        print("Bắt đầu xử lý video với tinh chỉnh kích thước Box...")

        while cap.isOpened() and frame_count < 10:
            ret, frame = cap.read()
            if not ret: break

            frame = cv2.resize(frame, (640, 480))

            # Nhận diện người
            (rects, weights) = hog.detectMultiScale(frame, winStride=(8, 8), padding=(3, 3), scaleUp=True)

            # TÍNH CHỈNH: Thu nhỏ box để ôm sát người hơn (HOG mặc định thường bỏ padding)
            for (x, y, w, h) in rects:
                # Thu hẹp chiều rộng và chiều cao khoảng 15-20%
                pad_w = int(w * 0.15)
                pad_h = int(h * 0.05)
                x_tight, y_tight = x + pad_w, y + pad_h
                w_tight, h_tight = w - 2*pad_w, h - 2*pad_h

                cv2.rectangle(frame, (x_tight, y_tight), (x_tight + w_tight, y_tight + h_tight), (0, 255, 0), 2)

            cv2.imshow("Video", frame)
            frame_count += 1

        cap.release()
        print(f"Hoàn tất xử lý {frame_count} frames.")
else:
    print("Tập video không khả dụng.")

```


Đang tải video mẫu...

Tải video thành công: sample_video.avi (8131690 bytes)

Bắt đầu xử lý video với tính chỉnh kích thước Box...

